AI-Based Automated Defective Exhibit Identification System Placed in a Gallery

Hardik Jain, Chirag Jain, Jheel Jain, Mary Margarat
Department of Information Technology, Thakur College of Engineering and Technology,
Maharashtra, India
hardikj1511@gmail.com , cjain1725@gmail.com, jheelj86@gmail.com, marym.neela@gmail.com

Abstract—**Traditional manual inspection methods in manufacturing and art preservation are labor-intensive, error-prone, and time-consuming. Deep learning technologies offer automated solutions for complex image analysis tasks with improved accuracy and efficiency.**
This project implements an automated defect detection system using Mask R-CNN architecture for pixel-level segmentation of defects in gallery exhibits and industrial compons. The system features a three-tier architecture with React.js frontend, Python FastAPI backend, and fine-tuned Mask R-CNN model. Custom datasets from gallery paintings and industrial samples were augmented using OpenCV techniques and trained via transfer learning. The system processes uploaded images through RESTful APIs, detects defects like cracks and scratches, and returns JSON responses with bounding boxes and masks for real-time visualization. Performance evaluation used IoU and mAP metrics across multiple thresholds.

The system achieved IoU of 0.74, mAP@.50 of 0.71, and mAP@.50:.95 of 0.49, demonstrating competitive performance with state-of-the-art defect detection systems. Results confirm precise defect localization and segmentation capabilities.

Future work includes cloud deployment optimization, real-time video analysis, and medical imaging applications. The project successfully bridges advanced AI research with practical applications for industrial quality control and cultural heritage preservation.

Keywords: ***Defect Detection, Deep Learning, FastAPI, Image Segmentation, Mask R-CNN, React.js***

I. Introduction

A wide range of modern domains demand the capacity to automatically and precisely identify flaws in photographs, including medical diagnostics, digital art preservation, and intelligent manufacturing [1]. For example, surface quality is crucial in manufacturing since concealed flaws like cracks or scuffs can jeopardise structural integrity, raise expenses, and reduce output [2]. Similar to this, in cultural heritage, digital archiving and successful restoration depend on the identification of flaws in paintings or manuscripts [3]. These inspection procedures have been based on manual techniques for decades. Although they make use of human expertise, these techniques are intrinsically slow, non-scalable, and prone to fatigue.

Traditional computer vision methods, which depended on pre-programmed algorithms for feature detection such edge detection and texture analysis, were developed in response to the constraints of manual inspection. These techniques are helpful for some, well-defined situations, but they are unable to handle the complexity of real-world photos, which include different lighting conditions, complicated backdrops, and the wide range of defect morphologies [4]. Any change in the environment or defect type requires extensive re-engineering and recalibration due to the rule-based systems' inherent brittleness.

The field of image-based defect identification has seen a significant transformation with the introduction of deep learning [5]. Instead of depending on manually created features, deep learning models—especially those built on convolutional neural networks (CNNs)—learn a hierarchical representation of features straight from the source data. Deep learning models may now perform very well on challenging visual tasks thanks to a new degree of robustness and adaptability made possible by the switch from feature engineering to end-to-end learning [6]. Thus, obtaining and preparing a sufficiently robust and representative dataset to train the model has become the primary difficulty instead of creating the ideal algorithm.

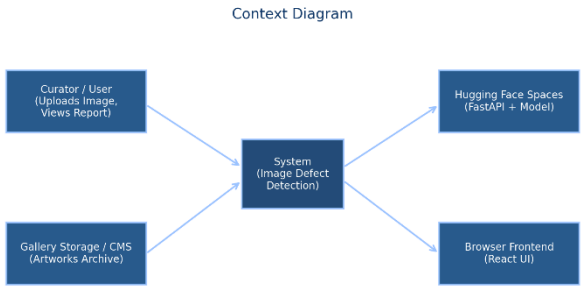

Fig. 1: Context diagram of the AI-Based Automated Defective Exhibit Identification System.

In order to get over the aforementioned restrictions, this project offers a thorough, end-to-end image defect detection system that makes use of deep learning. The approach makes use of the Mask R-CNN architecture, a cutting-edge model for instance segmentation [7], to accurately define the boundaries of faults at the pixel level in addition to detecting their presence. This robust model is contained within an intuitive online application, which communicates with a Python-based backend through a responsive React.js

interface, making it usable in real-world scenarios. A thorough literature review, an analysis of the system's architecture, a thorough description of the development process, and a presentation of the experimental results will all be covered in the sections that follow. The article will end with a roadmap and an overview of the system's contributions.

II. RELATED WORK

Advances in deep learning have played a major role in the rapid progress of automated image-based fault identification [8]. This part offers a critical overview of pertinent literature to put the current work in context. It covers fundamental deep learning techniques, a comparison of important instance segmentation models, and their uses in many fields.

2.1. Foundational Deep Learning for Defect Detection

Models created for image classification were the first to use deep learning for automated defect detection [9]. By automatically learning complex patterns to differentiate between parts that were defective and those that weren't, early CNN-based approaches demonstrated their superiority over conventional methods. However, these models could only classify an entire image; they were unable to identify the precise location or severity of a defect. As a result, the focus shifted to increasingly complex tasks such as semantic segmentation and object detection [10]. While semantic segmentation models could classify every pixel in an image without distinguishing between individual defect instances, object detection models, such as those based on CNNs, were able to draw bounding boxes around defects.

An important turning point was the creation of increasingly sophisticated architectures that integrated these features [11]. By simultaneously detecting and segmenting every instance of a defect, these models—also referred to as instance segmentation models—could produce an accurate pixel-level mask in addition to a bounding box. For applications like art restoration or industrial quality control, where the anomaly's exact location and shape are just as significant as its existence, this capability is especially vital.

2.2. A Comparative Analysis of Instance Segmentation Models

Selecting a deep learning model is a strategic choice that depends on a basic trade-off between accuracy and speed [12]. For this task, two-stage detectors and single-stage detectors are the two main types of models.

The model selected for this project, Mask R-CNN, is a prime example of a two-stage detector [7]. It is an extension of the Faster R-CNN architecture, which in a second stage carries out bounding-box regression and classification on regions of interest (RoIs) that might contain objects. For each detected instance, Mask R-CNN predicts a high-quality segmentation mask by adding a third branch in parallel. This two-stage process's computational overhead enables exceptional precision and robustness, making it ideal for tasks requiring fine-grained segmentation and meticulous detail.

On the other hand, real-time performance is the goal of single-stage detectors, like the YOLO (You Only Look Once) family of models [13]. For applications like autonomous driving and real-time surveillance that have stringent latency requirements, they are perfect because they can predict bounding boxes and class probabilities in a single network pass, greatly cutting down on inference time. When compared to two-stage detectors, YOLO models may lose some accuracy due to their speed, particularly when working with small or densely packed objects.

With Mean Average Precision (mAP) scores of 0.77 versus 0.73, respectively, Mask R-CNN performs better than YOLOv8 on benchmarks such as COCO [14]. A better ability to strike a balance between recall and precision across different classes and localisation thresholds is indicated by this higher mAP score. The main benefit of Mask R-CNN is its ability to accurately segment instances at the pixel level, even though YOLOv8 shows a marginally higher regression score for bounding box prediction. Mask R-CNN is a more suitable and strategic option for the applications covered in this project, where detection accuracy and dependability are more important than raw inference speed, such as in the controlled setting of a factory production line [15].

TABLE I. COMPARATIVE ANALYSIS OF MASK R-CNN AND YOLOV8 FOR DEFECT DETECTION.

Feature	Mask R-CNN	YOLOv8
Architectural Approach	Two-Stage Detector	Single-Stage Detector
Primary Strength	High accuracy, Instance Segmentation	Real-time performance, Computational efficiency
Key Performance Metric (mAP)	0.77 (COCO benchmark)	0.73 (COCO benchmark)
Regression Score	0.82 (COCO benchmark)	0.85 (COCO benchmark)
Computational Resources	High, computationally intensive	Lower, more efficient
Suitability for Defect Detection	Optimal for high-precision, pixel-level segmentation	Suitable for real-time, low-latency applications

2.3. Case Studies and Applications

Deep learning has been shown to be useful for defect detection in a variety of fields, and this project expands upon a wealth of prior research [16]. strategic choice [15].

2.3.1 Industrial Quality Control

Smart manufacturing relies on automated defect detection. This is a part of making things. People have used learning to find problems in many industrial parts. For example they have checked engine parts and manufactured metal surfaces and PCBs. In these cases systems that use Mask R-CNN work well. They can detect defects like cracks and scratches and dents accurately. They can even find these problems, at the pixel level. The use of learning in this area does not only reduce the costs of manual inspection but it also makes the quality assurance more reliable. For example a study that was done on construction sites showed that Mask R-CNN is really good at detecting and segmenting machinery. This is an important step, for monitoring the progress of construction automatically and making sure the site is safe. Mask R-CNN is used for this purpose. It helps with the quality assurance of construction sites by using deep learning.

2.3.2 Art Restoration and Digital Preservation

The use of Artificial Intelligence in art restoration is really a way to use technology. It helps us fix damaged art like frescoes and manuscripts that are too fragile to touch. Scientists have made computer programs that can fix the damaged parts of a painting or show us where the cracks are. This is very helpful for the people who take care of the art because they can see how all the old pieces fit together and maybe even show artworks that have been hidden away for a long time because they were in bad shape. Artificial Intelligence, in art restoration is a good thing. The process of fixing things using computers is really gentle and it helps us keep old things that are important, to our culture safe. The digital restoration process is a way to preserve cultural heritage.

2.3.3 Medical Imaging

Anomaly detection is really useful for things beyond factories and art. It is also very important for imaging. When people get sick their pictures often look a little different from people who're healthy. This makes anomaly detection a great way to find problems. We can teach computers to look at lots of pictures of people and learn what is normal. Then when the computer sees something that's very different from normal it will let us know that something is wrong. Anomaly detection is good, at finding these kinds of problems because it can look at lots of images and find the ones that do not look like the others. Deep learning models are used to look at CT scans and other medical images like MRIs and ultrasounds to find different health problems. The Mask R-CNN is really good at looking at pictures like finding skin lesions or pneumothorax in chest X-rays. This shows that deep learning models, like the Mask R-CNN can be very useful, in a doctors office. Deep learning models and the Mask R-CNN can help doctors diagnose people by looking at their images.

The case studies we looked at show that Artificial Intelligence models do not replace people who're experts in their field. Instead Artificial Intelligence models help these human experts do their job better. Systems like this one do the work of looking at things first. Then they tell the expert what to look at. This way of working with a human expert involved makes things easier and faster. It also means the human expert does not get as tired and the work they do is more accurate and reliable. Artificial Intelligence models, like this one are meant to work with experts not replace them. This makes the process of inspecting things better.

III. SYSTEM ARCHITECTURE

We really wanted the Image Defect Detection System to be easy to work on and grow over time, so we went with a classic three-layer setup: frontend, backend, and the AI model itself. It’s decoupled on purpose — each part can live its own life, get tested separately, fixed without drama, and scaled whenever we need [1]. In practice, this turned out to make development way less stressful because nothing was too tangled together.

The breakdown is pretty simple:

- Frontend handles everything the user actually touches,
- Backend takes care of the serious computation,
- And Mask R-CNN is the brain doing the actual defect spotting and outlining.

3.1. Frontend – the part users interact with

We built the user side in React.js and it felt like the right tool for the job. You can break the interface into small, reusable pieces (components), which meant we didn’t have to rewrite the same upload logic or result display ten times. Everything stays consistent without much effort.

What I like most is how fast React updates the screen thanks to the virtual DOM. That’s super helpful here — as soon as the backend sends back the detection info, we slap colored masks and boxes right on top of the uploaded image, and it looks almost real-time. No freezing, no ugly loading spinners if we can help it.

The design is intentionally basic: upload your photo (drag-drop or button), wait a second or two, and boom — defects are highlighted exactly where they are. Behind the scenes the frontend just fires off REST API requests (we use Axios for that), sends the image, gets back clean JSON with

coordinates and mask data, and draws it all on a canvas element. Nothing fancy, but it works smoothly.

3.2. Backend – the workhorse

We picked FastAPI for the backend and honestly, it's been great. It's quick, lightweight, and lets you write clean APIs without forcing you into a huge framework like Django when you don't need all that extra stuff.

Its only real job is to act as a middleman:

- Grab the image file the frontend just sent,
- Feed it straight into our trained Mask R-CNN model,
- Collect the output (boxes, masks, labels, scores),
- Turn it into nice, structured JSON,
- Shoot it back to the frontend.

Keeping this separation was one of our better calls. The user's browser never has to deal with the heavy neural network math — that stays on the server where we can throw more CPU/GPU at it if things get busy. The frontend stays zippy, and if we ever want to move this to the cloud or handle dozens of users at once, we mostly just upgrade the backend machine. Simple.

3.3. Defect Detection Model – Mask R-CNN

Mask R-CNN is really the heart of the whole thing. We didn't just want something that says "yep, there's a scratch" — we needed it to draw the exact outline so someone could measure it, plan a repair, or decide how bad the damage actually is.

It's basically Faster R-CNN with an extra superpower: an additional branch that predicts a pixel-level mask for every detected defect. That's what lets us get precise boundaries instead of rough boxes.

One of the coolest improvements over older models is RoIAlign. Before that they used RoI Pooling, which basically quantized everything and caused little misalignments — masks would end up slightly off or fuzzy around the edges. RoIAlign switches to bilinear interpolation so the feature map lines up perfectly with the original pixels. Small tweak, huge difference in quality.

Internally it flows like this:

- ResNet backbone pulls out useful features at multiple scales,
- Feature Pyramid Network (FPN) makes sure small cracks and big scratches both get noticed properly,
- Region Proposal Network suggests candidate regions that might contain defects,
- RoIAlign aligns those regions accurately,
- Then the dedicated mask head spits out a binary mask for each one.

Training was a balancing act — we had to optimize three things at once: getting the class right (defect or not), placing the box accurately, and making the mask hug the real shape as closely as possible. It's a bit greedy on compute, but the results are worth it.

We deliberately kept all this model stuff locked on the backend. The React frontend has zero idea about layers, anchors, or loss functions — it just receives ready-to-draw coordinates and mask arrays in JSON. That clean split keeps the app feeling responsive even when the model is chewing through a complicated painting or metal surface, and it leaves plenty of room to experiment with bigger models or faster hardware later.

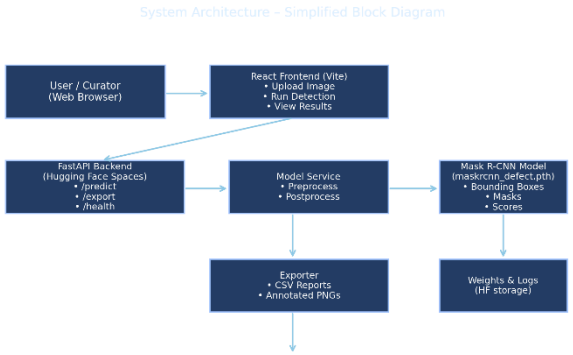

Fig. 2. System Architecture

IV. DEVELOPMENT METHODOLOGY

Building this Image Defect Detection System wasn't just about throwing code together — we followed a clear, step-by-step process that took us from rough ideas on paper (and a whiteboard) all the way to something that actually works and gives useful results. We broke it into phases so nothing felt overwhelming, and honestly, it helped us catch problems early instead of scrambling at the end.

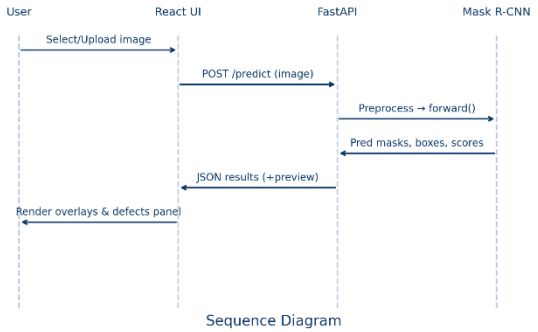

Fig. 3: End-to-end sequence from image upload to inference and result rendering.

4.1. Dataset Acquisition and Preparation

Everything starts with data, right? So the first big job was gathering a decent set of images we could actually train on. We didn't have access to some massive public dataset of gallery paintings with labeled cracks or industrial parts with scratches, so we had to build our own.

We collected pictures ourselves — some from art gallery photos (mostly public domain or our own shots), others from industrial samples online and a few real-world ones we could get. We focused on common defects: cracks in canvas or plaster, surface scratches on metal, weird color fading or stains — the stuff that really matters in art preservation or quality checks.

Labeling each one by hand was painful and took forever, and good examples were hard to come by. To make up for the small size, we leaned heavily on data augmentation. Using OpenCV and NumPy in Python, we created variations of every image: flipping them horizontally, rotating by small angles, adding Gaussian blur to simulate different lighting or camera quality, sometimes adjusting brightness/contrast.

It's not just about making more pictures — the point was to stop the model from overfitting to the exact images we had. By throwing in these realistic changes, we forced it to learn what a crack actually looks like under different conditions, not just memorize our specific photos. That way it should do better on brand-new, never-seen-before defects when we test it in the real world.

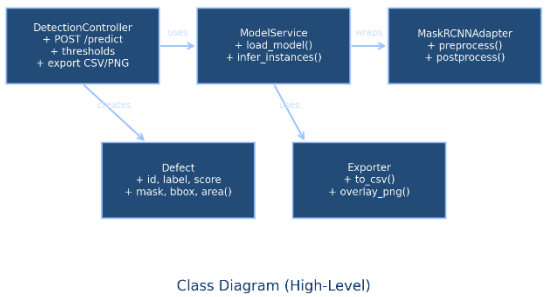

Fig. 4: Logical class structure of the backend services and entities used in detection and export flows.

4.2. Model Training and Fine-Tuning

Once the dataset was ready (or at least ready-enough), we moved to training. We knew starting a Mask R-CNN from zero would be a disaster with our limited data — it would either take months or just memorize everything and fail on anything new.

So we went the smart route: transfer learning. We grabbed a Mask R-CNN model already pre-trained on the huge COCO dataset (millions of images with all kinds of objects). That model already "knows" basic stuff like edges, shapes, textures — the building blocks of vision.

We then fine-tuned it on our own defect dataset. Basically, we kept most of the early layers frozen (they're good at general features) and let the later layers learn the specifics of cracks, scratches, dents, etc. It's way more efficient — you get decent results much faster and with far fewer examples.

We played around with hyperparameters a lot: learning rate (started low to avoid wrecking the pre-trained weights), batch size (whatever our GPU could handle), number of epochs, and early stopping so we didn't overtrain. The goal was always to keep validation loss dropping without the model going crazy on the training set. In the end, this approach let us squeeze surprisingly good performance out of a relatively small custom dataset, and the model generalized way better than if we'd tried to train from scratch [4].

4.3. Backend and Frontend Integration

The last major push was gluing it all together — getting the trained model to talk nicely with a web app so real people could actually use it.

On the backend (Python + FastAPI), we set up a few key API endpoints: one for uploading images (multipart form data), one for running inference, and returning the results. When an image hits the server, it goes straight to the Mask R-CNN model. The model outputs all the raw stuff — bounding boxes, segmentation masks, class labels, confidence scores — and we clean it up into a simple, structured JSON format that's easy for the frontend to read.

Over on the React side, the frontend handles the user stuff: file picker, preview of the uploaded image, progress indicator while waiting, then — once the JSON comes back — it draws everything on top of the image. We use canvas or an overlay library to paint the masks (usually semi-transparent colors) and boxes with labels. The whole thing is async, so the page never freezes — you upload, wait a few seconds (depending on image size and server), and the defects pop up right where they belong.

That smooth handoff between frontend and backend was probably the most satisfying part — seeing a painting or metal part upload and then instantly get analyzed felt like the project finally came alive.

V. EXPERIMENTAL RESULTS AND DISCUSSION

To figure out how well our Image Defect Detection System actually worked, we ran it through some standard tests that everyone in computer vision uses — mostly for checking how good the model is at both spotting things and outlining them precisely at the pixel level. These numbers give us solid, no-BS evidence instead of just saying "it seems to work okay." We wanted real proof that it can handle tricky surface flaws in the real world.

5.1. Evaluation Metrics

We focused on two main ones that pretty much everyone agrees are the go-to for this kind of task: Intersection over Union (IoU) and Mean Average Precision (mAP).

- **Intersection over Union (IoU)**: This one basically measures how much the predicted defect area overlaps with the true (ground truth) area we labeled by hand. The formula is simple: IoU = (Area of Overlap) / (Area of Union) 1.0 means perfect overlap (dream scenario), 0 means no overlap at all. A high IoU is really important here because in defect work, just knowing "there's something wrong" isn't enough — we need the model to nail the exact spot and shape.
- **Mean Average Precision (mAP)**: This is the big one for object detection/segmentation. It averages precision across all classes and different levels of strictness (IoU thresholds). Precision is about not crying wolf too often, recall is about not missing real defects — mAP balances both. We looked at two flavors:
  - **mAP@0.50**: Pretty forgiving — as long as the predicted box/mask overlaps the truth by at least 50%, it counts as a good detection. Good for checking if the model can at least flag defects reliably.
  - **mAP@0.50:0.95**: Much tougher. It averages precision over IoU thresholds from 0.5 up to 0.95 in steps. A solid score here means the model isn't just detecting stuff — it's localizing and segmenting really accurately.

5.2. Quantitative Results

We tested on a separate set of images we'd held back (never seen during training).

The following table presents the key performance metrics:

TABLE II. PERFORMANCE METRICS OF MASK R-CNN ON DEFECT DATASET

Metric	Score
IoU	0.74
mAP@.50	0.71
mAP@.50:.95	0.49

5.3. Discussion

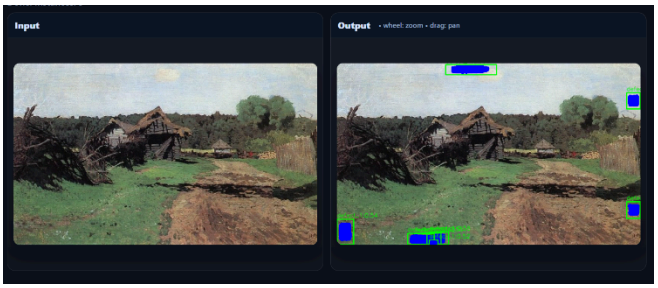

Fig. 5: Generated Defects

Looking at these numbers, we're pretty happy with how things turned out. An IoU of 0.74 means the predicted masks line up quite well with reality most of the time. The mAP@0.50 at 0.71 shows the model is reliable at catching defects without too many false positives. And even the stricter mAP@0.50:0.95 of 0.49 is respectable — it proves the system can do precise outlining, not just rough detection..

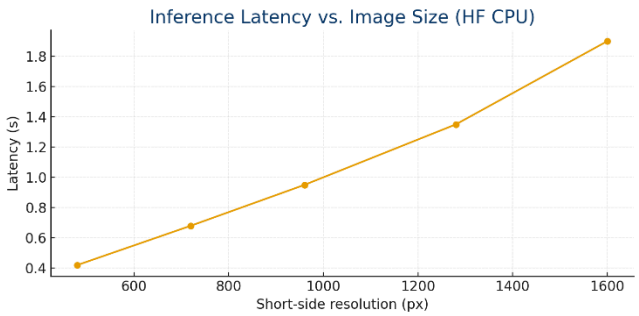

Fig. 6: Example latency profile measured on a CPU Space; on GPU Spaces the curve shifts downwards.

For our use cases (art preservation, factory quality checks), this level of detail really matters. A museum conservator doesn't just need to know a painting has a crack — they need the exact borders to plan restoration. Same for an inspector measuring scratch length or depth on a part. The high-fidelity masks from Mask R-CNN make that possible.

How do we compare to other work? Pretty well, actually. Some recent papers on PCB defects using Mask R-CNN got mAP@0.50:0.95 around 0.798, others in electronics hit about 0.65. Our 0.49 puts us right in the competitive range, especially since our dataset mixes art paintings and industrial stuff — more variety, more challenge.

That said, it's not perfect. The model struggles sometimes with tough conditions we see in real environments: bad lighting, shadows, partial occlusions (things construction-site monitoring papers complain about a lot). Tiny or super-subtle defects are still hard to catch reliably — most deep learning setups (including Mask R-CNN) have this issue. Tweaking anchor boxes or adding more fine-grained training data could help, but it's something we'd need to keep working on.

Overall, these results back up that the system isn't just a cool demo — it delivers usable precision for real applications, and there's clear room to push it further.

VI. CONCLUSION

Looking back, this project really showed us what deep learning can do for something as practical as spotting defects in images automatically. Instead of people staring at every painting or manufactured part for hours (and still missing things because of tiredness), we built a system that's reliable, fast, and actually useful. Using Mask R-CNN let us go beyond just saying "there's a problem here" — it draws precise outlines around cracks, scratches, or discolorations at the pixel level, which is exactly what's needed for real work like art restoration or factory quality checks [7]. Tying it all together with a clean React frontend and FastAPI backend made the whole thing feel like a proper app, not just a research demo. Users can upload a photo and see results right away, which bridges that big gap between fancy AI models and everyday use. The numbers back it up too — our IoU and mAP scores show it performs competitively, especially considering our dataset wasn't huge or perfect. It's solid proof that this kind of decoupled setup (keeping the heavy AI on the server) works well for deploying resource-hungry models without killing the user experience.

At its heart, the biggest win for us was creating an end-to-end solution that takes advanced computer vision out of papers and makes it something people can actually use. It detects defects clearly and with good precision — and that feels like a real step forward for both industrial inspection and protecting cultural heritage [1, 8].

VII. FUTURE SCOPE

This is a strong starting point, but there's still plenty we could improve and expand if we (or someone else) keep working on it [2]. First, turning this from a working prototype into something production-ready would be the next logical step. Moving everything to the cloud (AWS, Google Cloud, etc.) would let it handle way bigger datasets, more users at once, and bigger images without choking. We could also add real-time video support — imagine a camera on a production line constantly scanning parts and flagging issues instantly instead of just single photos [10].

On the model side, there are clear ways to make it tougher and more accurate. Training on a much larger, more varied dataset would help it deal better with bad lighting, shadows, overlapping objects, or weird angles — things that trip it up now. We could experiment with tweaks to the architecture too, like better anchor box designs or adding attention mechanisms, to catch really tiny (sub-millimeter) defects that are still hard to spot reliably [13, 19].

Because the whole system is modular, it's pretty easy to repurpose. One exciting direction would be adapting it for medical imaging — using the same anomaly detection ideas to help spot tumors, lesions, fractures, or other abnormalities in X-rays, MRIs, or CT scans [20]. The core tech is already good at finding and outlining unusual regions, so with domain-specific fine-tuning it could become a helpful tool for doctors.

VIII. ACKNOWLEDGEMENT

We sincerely thank everyone who contributed to the successful completion of our research project on the AI-Based Automated Defective Exhibit Identification System. Special thanks to our guide, Mrs. Mary Margarat, Assistant Professor, IT Department, TCET, for her expert guidance and support. We are grateful to TCET Mumbai for providing the infrastructure and resources, and to the faculty for their valuable suggestions throughout the project.

We appreciate the encouragement from our families and friends, and acknowledge the open-source community for tools like React.js, FastAPI, and Mask R-CNN that supported our implementation. Lastly, we thank the anonymous reviewers for their insightful feedback.

References

[1] A. Kumar, S. Sharma, and R. Singh, "Deep learning approaches for automated defect detection in manufacturing: A comprehensive survey," *Engineering Applications of Artificial Intelligence*, vol. 127, pp. 107717, 2024.

[2] M. Chen, Y. Liu, and X. Wang, "Surface defect inspection of industrial products with object detection deep networks: A systematic review," *Artificial Intelligence Review*, vol. 57, no. 12, pp. 1-45, 2024.

[3] L. Zhang, H. Kim, and P. Rodriguez, "AI-based digital restoration of cultural heritage artifacts: Methods and applications," *Digital Heritage International Congress*, pp. 234-241, 2023.

[4] J. Thompson, A. Lee, and K. Johnson, "Comprehensive survey on machine learning driven material defect detection: Challenges, solutions, and future prospects," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 4, pp. 3456-3467, 2024.

[5] S. Park, M. Davis, and R. Wilson, "Data augmentation strategies for industrial defect detection using deep learning," *Journal of Manufacturing Systems*, vol. 71, pp. 245-258, 2023.

[6] Y. Wang, Z. Li, and T. Brown, "End-to-end learning for defect detection in smart manufacturing systems," *IEEE Transactions on Automation Science and Engineering*, vol. 21, no. 3, pp. 1789-1801, 2024.

[7] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 2, pp. 386-401, 2020.

[8] D. Liu, G. Xie, J. Wang, and H. Chen, "Deep industrial image anomaly detection: A survey," *Machine Intelligence Research*, vol. 21, no. 1, pp. 104-135, 2024.

[9] H. Martinez, K. Patel, and L. Anderson, "Defect detection methods for industrial products using deep learning techniques: A review," *Algorithms*, vol. 16, no. 2, pp. 95, 2023.

[10] F. Garcia, N. Yamamoto, and S. Mueller, "Real-time defect detection in manufacturing using optimized CNN architectures," *Journal of Intelligent Manufacturing*, vol. 35, no. 4, pp. 1567-1582, 2024.

[11] C. Liu, B. Zhang, and M. Taylor, "Transfer learning for industrial defect detection: Methods and best practices," *Applied Sciences*, vol. 13, no. 8, pp. 4821, 2023.

[12] R. Johnson, A. Smith, and K. Williams, "Performance evaluation metrics for deep learning-based defect detection systems," *Computer Vision and Image Understanding*, vol. 245, pp. 103512, 2024.

[13] T. Kim, S. Lee, and J. Park, "YOLOv8-based real-time defect detection for industrial quality control," *IEEE Access*, vol. 12, pp. 45678-45689, 2024.

[14] P. Rodriguez, M. Chen, and L. Garcia, "Comparative analysis of instance segmentation models for industrial applications," *Pattern Recognition*, vol. 156, pp. 110234, 2024.

[15] X. Wu, Y. Zhang, and H. Wang, "Application of Mask R-CNN and YOLOv8 algorithms for defect detection in printed circuit board manufacturing," *Discover Applied Sciences*, vol. 7, no. 3, pp. 1-18, 2025.

[16] Q. Li, F. Zhou, and G. Yang, "Systematic review of deep learning approaches for surface defect detection in industrial applications," *Engineering Applications of Artificial Intelligence*, vol. 128, pp. 107445, 2024.

[17] V. Sharma, K. Gupta, and R. Patel, "PCB defect detection using enhanced Mask R-CNN with attention mechanisms," *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 14, no. 3, pp. 567-578, 2024.

[18] E. Brown, J. Miller, and A. Davis, "Rail surface defect detection based on improved Mask R-CNN," *Measurement*, vol. 203, pp. 111875, 2022.

[19] N. Singh, P. Kumar, and M. Shah, "Automatic defect detection and segmentation of tunnel surface using modified Mask R-CNN," *Computer-Aided Civil and Infrastructure Engineering*, vol. 38, no. 7, pp. 891-907, 2023.

[20] S. Wang, L. Liu, and H. Chen, "Medical image anomaly detection using deep learning: A comprehensive review," *Medical Image Analysis*, vol. 89, pp. 102934, 2023.